AAS Time Off Planner: Technical Requirements Document

Version 1.0 draft for review

23 October 2026

1. Document control

Item	Detail
Document	Technical Requirements Document (TRD)
Version	1.0, draft for review
Date	23 October 2026
Companion	Product Requirements Document (PRD) v1.0; requirement IDs FR- and NFR- refer to it
Code	`github.com/SeanPodnar/AAS_Time_Off_Planner_V1`, folder `app/`
Built so far	Migrations 001 to 003: companies, people, audit log, company data separation, email sign-in, sessions, roles, platform owner sign-in, two-step sign-in. 60 automated tests.

2. Architecture

```
Browser (employee, manager, HR admin, platform owner)
  | HTTPS: name.<platform domain> or a custom domain
  v
Load balancer: TLS, sets X-Forwarded-Host/For
  |                                    [AWS Canada Central]
  v
Next.js app (Node 22, TypeScript)
  |  server components and server actions
  |  background jobs: accruals, year end, reminders,
  |  payroll files, calendar push, webhooks
  |  two database roles, never superuser
  v
PostgreSQL 16, row-level security on company tables
  |
  +-- backups and point-in-time recovery,
      copies in AWS Canada West

Email: Amazon SES (Canada)    Files: S3 (Canada)
Secrets: AWS Secrets Manager
```

One application serves every company. The request's host decides the company. The database, not the application code, enforces separation between companies.

3. Technology stack

Layer	Choice	Version (pinned)	Reason
Runtime	Node.js	22 LTS	Long-term support
Web framework	Next.js (App Router), React	15.5.28, 19.1.10	Server rendering, server actions with built-in origin checks
Language	TypeScript, strict	5.9.3	
Database	PostgreSQL	16	Row-level security, JSONB, mature managed options in Canada
Database driver	postgres.js, plain SQL migrations	3.4.9	Separation rules readable as SQL; no ORM layer between app and policies
Validation	Zod	4.6.7	Upgraded for advisory GHSA-mrhx-c7xc-m3w5
QR codes	qrcode	1.5.4	Two-step set-up
Tests	Vitest	5.0.5	Unit and real-database tests
CI	GitHub Actions with a Postgres 16 service		Audit, lint, type check, tests, build on every pull request

Dependency policy: exact versions, `npm audit --audit-level=high` blocks merges, PostCSS forced to a patched version through `overrides` while on Next.js 15.

4. Company data separation (NFR-01)

4.1 Database roles

Role	Can	Cannot
Admin (owner)	Run migrations only	Is never used by the running app
`aas_app`	Read and write company tables, only for the company set on the current transaction; look up one company's public branding by host	Read the companies table; read platform tables; bypass row-level security
`aas_platform`	Manage companies and platform owner sign-in tables	Read any company table (people, sessions, audit log and so on)

4.2 Row-level security

- Every company table has `tenant_id`, row-level security **enabled and forced**, and a policy `tenant_id = app_tenant_id()` for both reading and writing.
- `app_tenant_id()` reads the transaction setting `app.tenant_id`. With nothing set it returns NULL, so queries return no rows (fails closed).

- The app reaches company data only through `withTenant(id, fn)`, which opens a transaction and sets the company with `set_config(..., true)`, so the setting ends with the transaction and cannot leak to the next request on a pooled connection.
- Links between rows use composite foreign keys `(tenant_id, id)`, so a manager, session or factor can never point at another company's person, even for the admin role.
- A guard test lists every table with a `tenant_id` column and fails if any lacks forced row-level security and an `aas_app` policy.

4.3 Moving a company to its own database later

UUID keys everywhere (no shared sequences), no cross-company foreign keys, files under a per-company prefix. A tenant registry will map company to connection string; moving a company is copy, verify counts and checksums, switch the registry entry, delete from the shared database.

5. Data model

5.1 Built (migrations 001 to 003)

Table	Purpose	Key columns
`tenants`	Companies	legal_name, short_label (unique ignoring case and spaces, 24 max), subdomain, custom_domain, brand_color, country, status
`people`	Employees	tenant_id, full_name, email (unique per company), job_title, jurisdiction (`CA-ON`, `US-CA`), hire_date, manager_id, role, status
`audit_log`	Append-only history	tenant_id, at, actor, action, detail (JSON). Update and delete blocked by trigger and grants.
`login_tokens`, `sessions`	Email sign-in	SHA-256 token hashes, expiry, used or revoked time, method, mfa_at
`mfa_factors`, `mfa_recovery_codes`	Two-step sign-in	Encrypted key, confirmed time, last used step; hashed recovery codes
`platform_owners`, `platform_login_tokens`, `platform_sessions`, `platform_mfa_factors`, `platform_mfa_recovery_codes`	Platform owner sign-in	Reachable only by `aas_platform`

5.2 Planned for v1

Table	Purpose
`tenant_settings`	Time zone, second-approval threshold (default 10), retention, sign-in methods allowed, allowed email domains
`teams`, `locations`	Grouping and holiday calendar selection

Table	Purpose
person_managers	Secondary managers (primary stays on people.manager_id)
work_schedules	Standard hours per day and work days per person (pending decision D3)
plans, plan_levels, plan_assignments	Plans, levels with days per year, one level per person per plan, effective dates
accrual_ledger	Every credit and debit as a row: grant, accrual, proration, carryover, forfeiture, override, adjustment, booking. Balances are sums, never stored totals, so every number can be explained.
jurisdiction_rules	Versioned, dated statutory rules: minimum entitlement by years of service, forfeiture rule, sick leave minimums and caps, by province, state and city
holidays	Per jurisdiction and per company
requests, request_days, approval_steps	Requests, the working days they cover (with half days), and each approval step
calendar_feeds	Hashed feed tokens per person and team
payroll_exports, payroll_codes	Pay period files and plan to WorkLinks code mapping
notifications_outbox	Emails queued in the same transaction as the change that caused them
jobs	Background job queue (Postgres SKIP LOCKED)

6. Request routing and branding

- parseHost(host, rootDomain): the bare root is the platform console; one subdomain level is a company; any other host is looked up as a custom domain. Look-alike domains (evilaas-timeoff.ca) are not treated as subdomains.
- The company is resolved through the security-definer function resolve_tenant_by_host, which returns only id, short label, brand colour and country for non-suspended companies.
- The host is read from X-Forwarded-Host first. Next.js renders a server action's redirect with Host set to its own address, and in production the load balancer sets the header. A forged value only selects which company's records are checked; it must match a company, and sessions are looked up inside that company, so it cannot reach another company's data. The load balancer must overwrite, not append, client-supplied values.
- Brand colour drives CSS variables; button text colour is chosen by contrast (WCAG formula).

7. Sign-in and sessions

7.1 Email link (built)

1. The person enters an email on their company's address. Rate limits: 5 per address, 30 per network address per 15 minutes.

2. If the address belongs to an active person in that company, a 256-bit random token is stored as a SHA-256 hash with a 15-minute expiry and emailed. The answer is identical either way.
3. The link opens a Continue page; the POST redeems the token once and creates a session.

7.2 Sessions (built)

256-bit token, stored hashed, 12-hour expiry, cookie `HttpOnly`, `SameSite=Lax`, `Secure` in production, host-only (no `Domain` attribute) so a session never travels to another company's address. Sign-out revokes the row. Server actions get Next.js's origin check against cross-site requests.

7.3 Two-step sign-in (built)

RFC 6238 codes (SHA-1, 6 digits, 30 seconds, one step of drift either way), verified against the RFC test vectors. The key is encrypted with AES-256-GCM using `MFA_ENCRYPTION_KEY` with the person's id as associated data. The last used step is stored so a code works once; recovery codes are hashed and single use; 5 wrong codes per session lock it for 15 minutes. Required for HR admins and the platform owner; checked on every protected page.

7.4 Google and Microsoft 365 (next)

- OpenID Connect authorization code flow with PKCE, state and nonce.
- Google rejects wildcard return addresses, so all companies share one fixed return address on the root domain: `https://<root>/auth/<provider>/callback`. The start request carries the company in signed state. After the ID token is verified, the app creates a 60-second single-use hand-off code and redirects to the company's own address, which exchanges it for a session cookie on that host.
- Microsoft uses the multi-tenant (`organizations`) endpoint; the issuer is checked against the token's tenant id. Optionally each company can restrict sign-in to its own Google Workspace domain or Microsoft tenant id.
- The email in the token must be verified and must match an active person in that company. No accounts are created from sign-in.
- Secrets: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET` in the secret store. Microsoft secrets expire within 24 months; a reminder job warns 30 days ahead.

8. Accrual engine (FR-30 to FR-42)

- Ledger-based: every balance movement is a dated row in `accrual_ledger`. A balance at any date is the sum of rows up to that date plus projected future accruals. Reports and the request form use the same function.
- Accrual job runs nightly per company at 01:00 company time and is idempotent (each row has a unique key per person, plan, period and kind), so reruns never double-credit.
- Proration: remaining months including the start month if the start date is on or before the 15th; rounded per plan (default nearest half day; hours plans to 0.25 h).
- Hours plans: salaried staff accrue on their standard week (default 40 hours); requests deduct standard hours per day per day booked.
- Year end: a job 30 days ahead builds the preview; at midnight on the plan year boundary it applies the plan rule, then jurisdiction overrides (`jurisdiction_rules` effective at that date), writing each step as ledger rows with the rule id that caused it.

- Statutory checks are pure functions over the rules table: on plan save, nightly for anniversaries within 90 days, and in the compliance report.
- Working day count: dates between first and last day, minus weekends (from the person's work schedule), minus holidays for the person's jurisdiction and company, with half days on the first and last day.

9. Approval workflow (FR-50 to FR-57)

- On submit or change, the server computes the run: the request plus the person's live bookings (any plan, approved or pending) that touch it with no working day between. If the run exceeds the company threshold and the plan has a second step, steps are primary manager then next-level manager, falling back to HR admin.
- Steps are rows in `approval_steps` with status waiting, pending, approved or declined. Only the current step's approver (or an HR admin, logged as override) can decide.
- Change requests keep the original booking in force until approved; cancellation of approved future time follows the plan's setting.
- All writes for one decision (step, request status, ledger booking, notification outbox, audit entry) happen in one transaction.
- Reminder job: 2 business days pending sends a reminder; 5 escalates to HR (configurable).

10. Calendars

- ICS feeds (v1): one hashed token per person and per team, resettable. Events carry name, dates, half-day marker and status (Busy, Free, Out of office). Private plans show "Out of office". Feeds set `Cache-Control: private` and an ETag; refresh timing is controlled by the calendar app.
- Direct push (v1.1): per-person OAuth to Google Calendar API and Microsoft Graph; events written from the outbox within seconds; tokens encrypted like two-step keys.

11. Integrations

Integration	Design
WorkLinks (v1)	Pay period file per company in WorkLinks' import layout (to be confirmed, D5). Approved absences by payroll code; corrections for periods already sent are separate rows. Download or SFTP.
WorkLinks API (blocked)	Same mapping and outbox; transport swapped once documentation exists.
Jira (v1.1)	Per-company OAuth 2.0 (3LO) to the company's Jira Cloud site, least-privilege scopes. Absence issue per approved absence with dates and original estimate; updated or cancelled from the outbox.
Availability API (v1.1)	`GET /api/v1/availability?email&from&to` returns per-day out or available with half days. Per-company API keys (hashed, scoped, rotatable, rate limited, optional IP allow list). Webhooks signed with HMAC-SHA256 and a timestamp.
MCP server (backlog)	Tools for balances, requests, who's out, availability, request and approve; OAuth per person; same authorisation and privacy code as the web app.

12. Reports and exports

- Server-side queries over the ledger and requests, always inside `withTenant`; HR admin only.
- XLSX via a streaming writer with the run label in the first rows, frozen header, auto-filter, number formats with units. CSV in UTF-8 with BOM for Excel.
- Every export writes an audit entry with filters used.

13. Email

- Development: emails print in the terminal and go to an in-memory outbox for tests.
- Production: Amazon SES in Canada (Central), sending domain with SPF, DKIM and DMARC. Emails are queued in `notifications_outbox` in the same transaction as the change and sent by a job with retries.
- Subjects start with the company short label. Bodies avoid private plan names except to the person, their approvers and HR.

14. Security controls (NFR-04)

Control	Status
Separation by database roles and forced row-level security, with guard test	Built
Tokens and recovery codes stored as SHA-256 hashes; two-step keys AES-256-GCM	Built
Host-only HttpOnly cookies; origin-checked server actions	Built
Rate limits on sign-in links and codes	Built (in memory; see 19)
Security headers: nosniff, frame deny, strict referrer, permissions policy	Built
Content Security Policy with nonces	Planned v1
Dependency audit on every pull request	Built
Secrets in AWS Secrets Manager; no secrets in the repository	Planned for release (development uses throwaway values)
Encryption at rest (RDS, S3) and TLS 1.2+ everywhere	Planned for release
Audit log append-only, including platform owner visits	Built (visits pending)
Penetration test and review against OWASP ASVS level 2 before real data	Planned (board item)

15. Hosting, environments and cost

Phase	Where	Data
Build (now)	Local and CI; optional free Supabase project in Canada (Central)	Synthetic only
Release	AWS Canada (Central), Montreal: app on ECS Fargate or App	Real

Phase	Where	Data
	Runner, RDS PostgreSQL 16 with point-in-time recovery, SES, S3, Secrets Manager; backup copies in Canada West, Calgary	

Estimated release cost for 4 companies and under 100 people: about US$60 to US$110 a month (smallest RDS instance with backups about $20 to $30, one small app container $15 to $30, load balancer about $20, the rest a few dollars). Confidence: moderate; to be priced with the AWS calculator at release.

Environments: development (from `main`, synthetic data), production (from release tags, after the security review).

16. Delivery process

- Trunk-based: `main` always green; one branch per board item; pull request with the template; CI must pass; owner approves.
- Releases tagged `v1.0.0`, `v1.1.0`; hotfixes `v1.0.x`.
- GitHub branch protection is configured but not enforced on the free plan for a private repository; GitHub Pro (about US$4 a month) enforces it.

17. Testing strategy

Level	What	Where
Unit	Validation, host parsing, contrast, tokens, rate limits, TOTP (RFC vectors), encryption, accrual and day-count functions	Every PR
Database	Separation tests (cannot read, insert, move, update, delete across companies; no cross-company links; no leak across pooled connections; role limits; guard test), sign-in and two-step flows against Postgres 16	Every PR
End to end	Browser runs of each role's main flows (Playwright)	Each feature PR; automated in CI in v1
Accrual golden tests	Worked examples per province and state checked by the legal reviewer	Before release
Security	Dependency audit every PR; penetration test before release	

18. Observability and operations

Structured logs without personal data beyond ids; error tracking hosted in Canada or self-hosted; health check endpoint; alerts on job failures, email bounces and sign-in spikes; daily backup restore test in the first month, monthly after.

19. Known limitations and technical debt

Item	Plan
Rate limits are in memory (one server)	Move to Postgres when running more than one instance
Production email transport not wired	SES with the release item
Next.js 15 with PostCSS override	Move to Next.js 16 when stable for our needs; remove override
`X-Forwarded-Host` trusted	Load balancer must overwrite the header; documented in deployment checklist
Branch protection not enforced	GitHub Pro
Two-step not offered to employees	Decision D8

20. Open technical decisions

#	Decision	Recommendation
T1	App hosting on AWS: ECS Fargate or App Runner	App Runner for simplicity if available in Canada (Central) at release; otherwise Fargate
T2	Background jobs: Postgres queue or a managed queue	Postgres `SKIP LOCKED` queue (one less service)
T3	Error tracking with Canadian residency	Self-hosted or a vendor with a Canadian region
T4	Work schedules for part-time staff (PRD D3)	Include in v1: work days and hours per day per person
T5	Company time zone (PRD D4)	One per company

21. Traceability

Requirement	Section
FR-01 to FR-06	4, 5, 6
FR-10 to FR-15	7
FR-20 to FR-22	5.2
FR-30 to FR-42	8
FR-50 to FR-57	9
FR-60 to FR-63	10
FR-70 to FR-80	12
FR-90 to FR-95	11
FR-100	13
FR-110	4, 14
NFR-01 to NFR-11	4, 14, 15, 17, 18